All the modifiers you've seen so far, such as and_return and and_raise, as well as ones you haven't seen, such as with, can be added to a mock just like they can to a stub.

By default, the expects method sets a validation that the associated method is called exactly once during the test. In case that does not meet your testing needs, RSpec has methods that let you specify the number of calls to the method. These methods are largely self-explanatory:

```
proj = Project.new
expect(proj).to receive(:name).once
expect(proj).to receive(:name).twice
expect(proj).to receive(:name).at_least(:once)
expect(proj).to receive(:name).at_least(:twice)
expect(proj).to receive(:name).at_least(n).times
expect(proj).to receive(:name).at_most(:once)
expect(proj).to receive(:name).at_most(:twice)
expect(proj).to receive(:name).at_most(n).times
expect(proj).not_to receive(:name)
```

In practice, the default behavior is good for most usages (I'd worry if I started needing these decorators a lot), though not_to is sometimes useful to guarantee that a particular expensive method is not called. Note that a stub is equivalent to a mock expectation defined with at_least(0).times.

Using Mocks to Simulate Database Failure

A common use case for test doubles in Rails is to simulate database failures. Let's look at how you might do that.

In the past, I would have suggested testing for failure at the controller level. However, Rails has basically deprecated controller tests for reasons I'll talk about in Chapter 11, *Testing Rails Display Elements*, on page 217, and so I'd like to stay within the bounds of Rails community behavior.

Let's look at the project-creation functionality you wrote at the beginning of the book. You wrote a controller:

mocks/01/app/controllers/projects_controller.rb

```
def create
  @workflow = CreatesProject.new(
    name: params[:project][:name],
    task_string: params[:project][:tasks])
  @workflow.create
  if @workflow.success?
    redirect_to projects_path
```